

SIT720 8.1D - Distinction Task

Github - <https://github.com/Hassan-Data-Dev/8.1D.git>

Introduction and Approach

An agency needs an indicative price before a formal appraisal. This project delivers that end to end: real sold-property data for three contrasting Sydney suburbs, feature engineering, three models compared under cross-validation, failure analysis, a benchmark against an LLM and human judgement, and a deployed web application.

Part 1: Problem Definition and Data Collection

The target is sale price. 105 genuinely sold properties were collected, 35 in Mosman, 35 in Parramatta and 35 in Campbelltown, covering sales from 2026-05-22 to 2026-09-12.

The three are suburbs I would personally consider buying in, chosen because they are genuinely different markets rather than three versions of one. Mosman is premium harbourside, where land scarcity and amenity dominate. Parramatta is the second CBD and almost entirely apartments, where the building and its transport access matter more than the site. Campbelltown is affordable and outer-metropolitan, with houses and units on larger lots. Because those buyer groups value different things, the same feature should carry different weight in each suburb.

Median prices confirm the contrast: \$1,310,000 in Mosman, \$590,000 in Parramatta and \$637,595 in Campbelltown.

Suburb	Records	Median price	Dominant dwelling	Role in analysis
Mosman	35	\$1,310,000	Apartment	Premium harbourside
Parramatta	35	\$590,000	Apartment	Second CBD, transport-driven
Campbelltown	35	\$637,595	Mixed	Affordable, family market

Collection read sold listings on domain.com.au with a browser-assisted script, disclosed here rather than presented as wholly manual: the inclusion rules were specified by hand and every excluded listing is logged with its reason. Each row was then re-opened against its own source URL and its address, suburb, postcode, sale date, room counts and disclosed price compared with the page; the per-property outcome is in the submitted verification report. realestate.com.au blocked automated access and the NSW open-data and Valuer General sales datasets were unavailable, so one source had to suffice and prices could not be cross-checked against a second.

Three limitations shape everything that follows. **Prices are frequently withheld**: only about one in ten listings disclosed one, so the sample is the minority of sales sellers chose to publish. **Land size and floor area are unavailable** for sold listings, removing what would plausibly have been the strongest predictor of house value. And the sample is small, so a few unusual sales shift a whole suburb's apparent level.

Part 2: Data Understanding and Feature Engineering

Sale price is strongly right-skewed (3.80), spanning \$320,000 to \$11,650,000. All models are therefore trained on the log target so that a few very expensive properties do not dominate the loss.

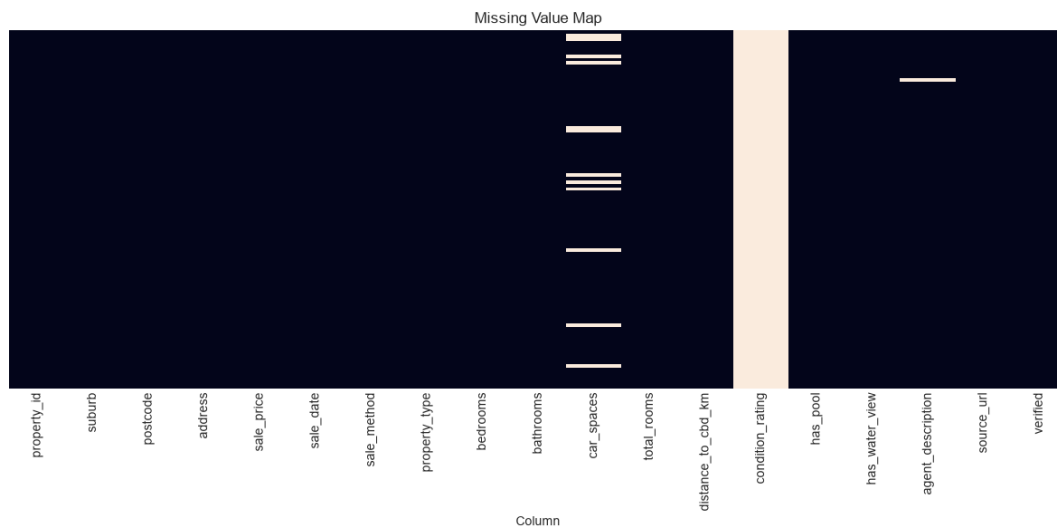


Figure 1: Missing value map.

Two absences are visible: car spaces, blank where a listing states none, and **condition rating, blank on every row**. Condition is a judgement from photographs and cannot be read from the listing record, so it was left empty rather than guessed; the column remains because it is the field a valuer would most want, and it is treated as unobserved in modelling.

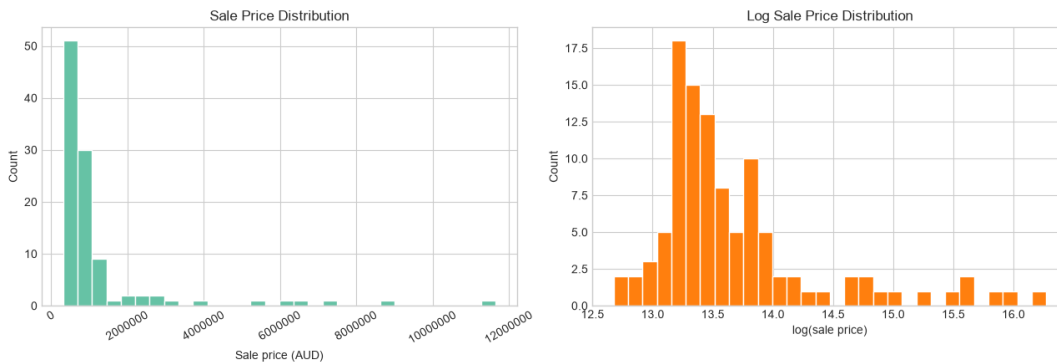


Figure 2: Sale price and log sale price distributions.

The suburbs separate as intended, and their dispersion differs sharply in dollar terms, so RMSE will be dominated by the widest-spread suburb. That is why MAE and percentage error are reported alongside it throughout.

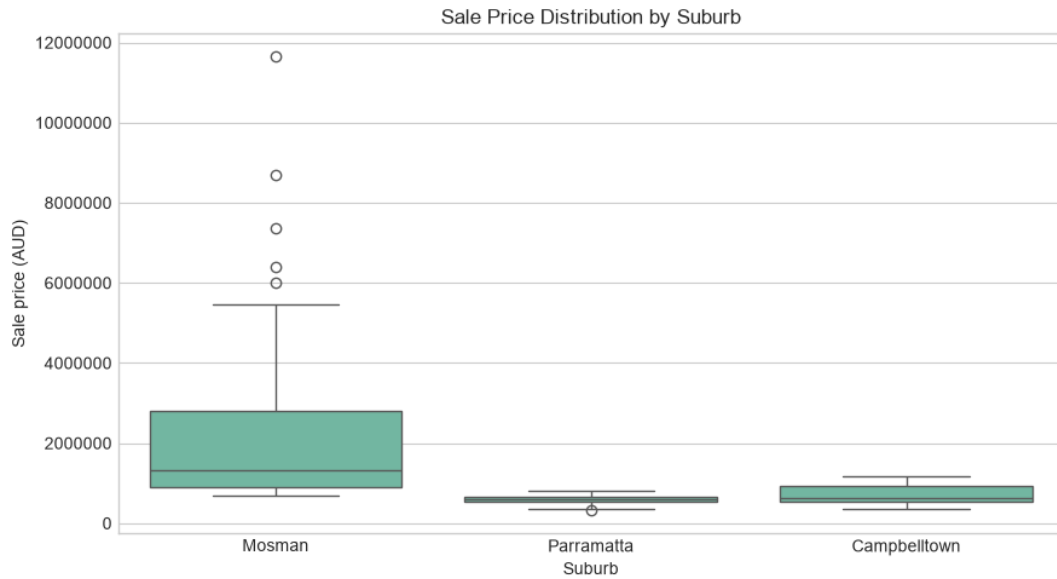


Figure 3: Sale price distribution by suburb.

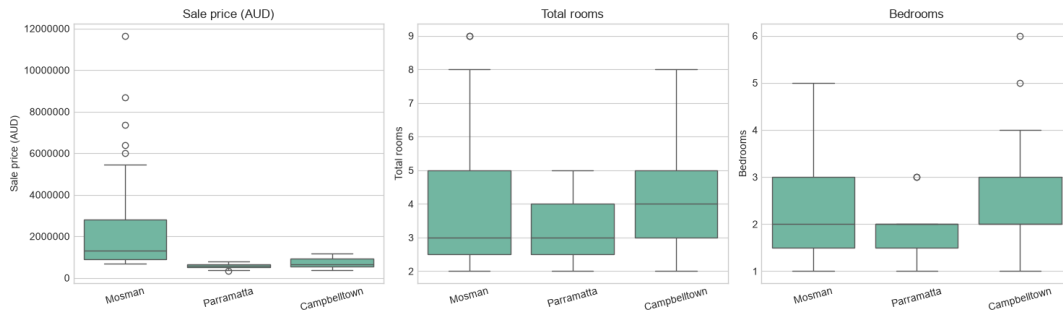


Figure 4: Price, rooms and bedrooms by suburb.

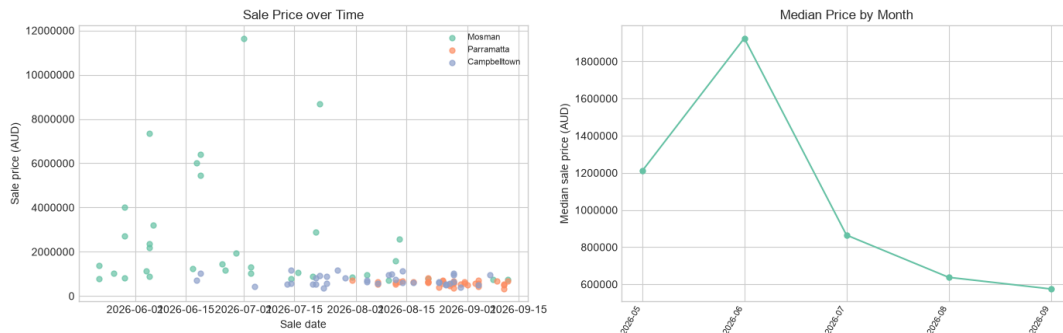


Figure 5: Price over time and median price by month.

Outlier screening flagged 14 record(s) by the IQR rule. These were retained rather than removed: in property data the extreme values are genuine prestige sales the agency would need priced, not errors. They reappear as the largest failures in Part 4.

The three variables expected to matter most

Recorded before any feature was created: **property type, distance to the CBD, and dwelling size proxied by room count**. Property type was expected to dominate because it separates two different price processes, land-driven houses and building-driven apartments. Distance was expected to capture the location premium. Room count was expected to be the only usable size signal, since neither land nor floor area is observable.

Feature engineering added room totals, keyword flags from the verbatim agent descriptions, and a description-length indicator. Text coverage was 99%, so the text features are available for almost every row.

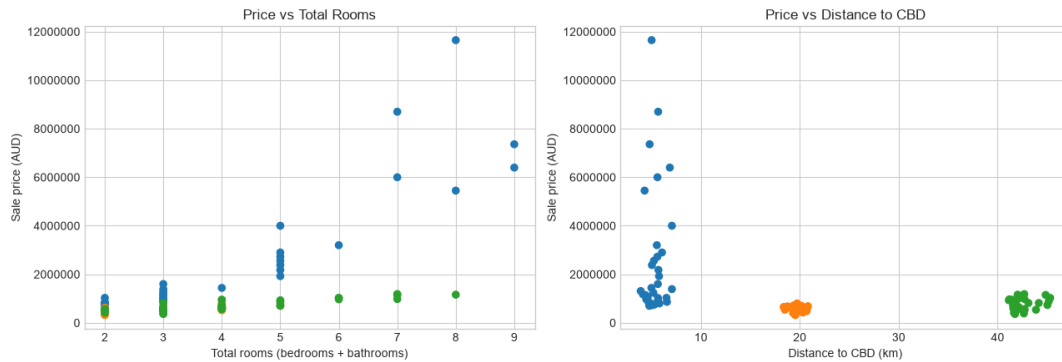


Figure 6: Price against room count and distance to the CBD.

One expectation needed revision rather than confirmation. Dwelling size is represented only by room count, because land size and floor area do not exist in this dataset. A room count cannot distinguish a small house on a large lot from a large house on a small one, so the size signal is weaker than intended. That is a limitation of the source, not of the feature engineering.

Part 3: Model Development and Evaluation

Three models spanning different paradigms were compared. **Ridge regression** was the regularised linear baseline: interpretable, but unable to express the interaction between dwelling type and the other predictors. **Random Forest** captures non-linear effects automatically and resists extreme values, but interpolates rather than extrapolates. **Histogram Gradient Boosting** is often most accurate on tabular data but most prone to overfitting here.

The expectation recorded before training was that **Random Forest would win**, Ridge would be competitive on MAE because it resists the extreme prices, boosting would be weakest, and the gaps would be modest. Models were evaluated by five-fold cross-validation, with the test set touched once at the end. The recommended model is **RandomForest**.

Model	CV RMSE	CV MAE	CV R2	CV MAPE
Ridge	\$945,493	\$429,596	0.731	24.3%
RandomForest	\$765,319	\$407,538	0.815	27.8%

HistGradientBoosting	\$1,101,432	\$492,170	0.645	25.8%
-----------------------------	-------------	-----------	-------	-------

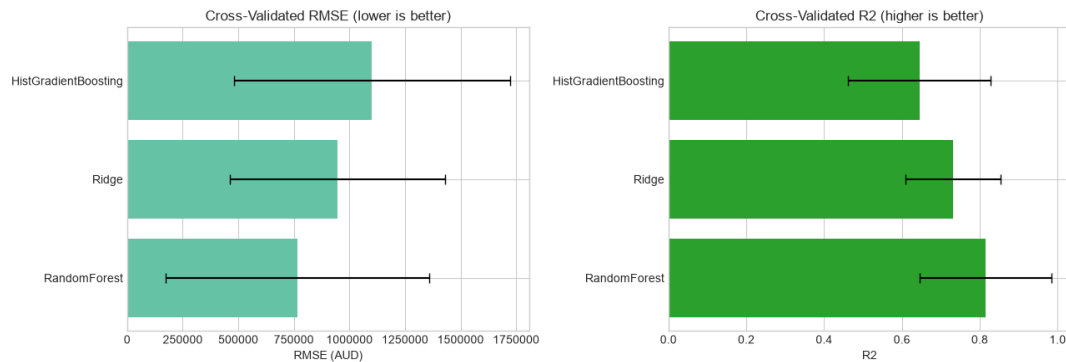


Figure 7: Cross-validated RMSE and R2 by model.

The expectation was largely supported, with one qualification. Random Forest ranked first on cross-validated RMSE and R2 and boosting was weakest, as predicted. But cross-validated errors far exceed the test-set error, and the reason is instructive: one property sold several million dollars above anything comparable, and whichever fold contained it dominated that fold's error.

Model	Test RMSE	Test MAE	Test R2	Test MAPE
Ridge	\$213,723	\$155,019	0.907	18.7%
RandomForest	\$250,415	\$154,650	0.873	17.9%
HistGradientBoosting	\$512,001	\$291,924	0.468	27.9%

The selected model achieved a test RMSE of \$250,415, MAE of \$154,650, R2 of 0.873 and MAPE of 17.9%. For the agency, the typical prediction is within roughly 17.9% of the actual sale price.

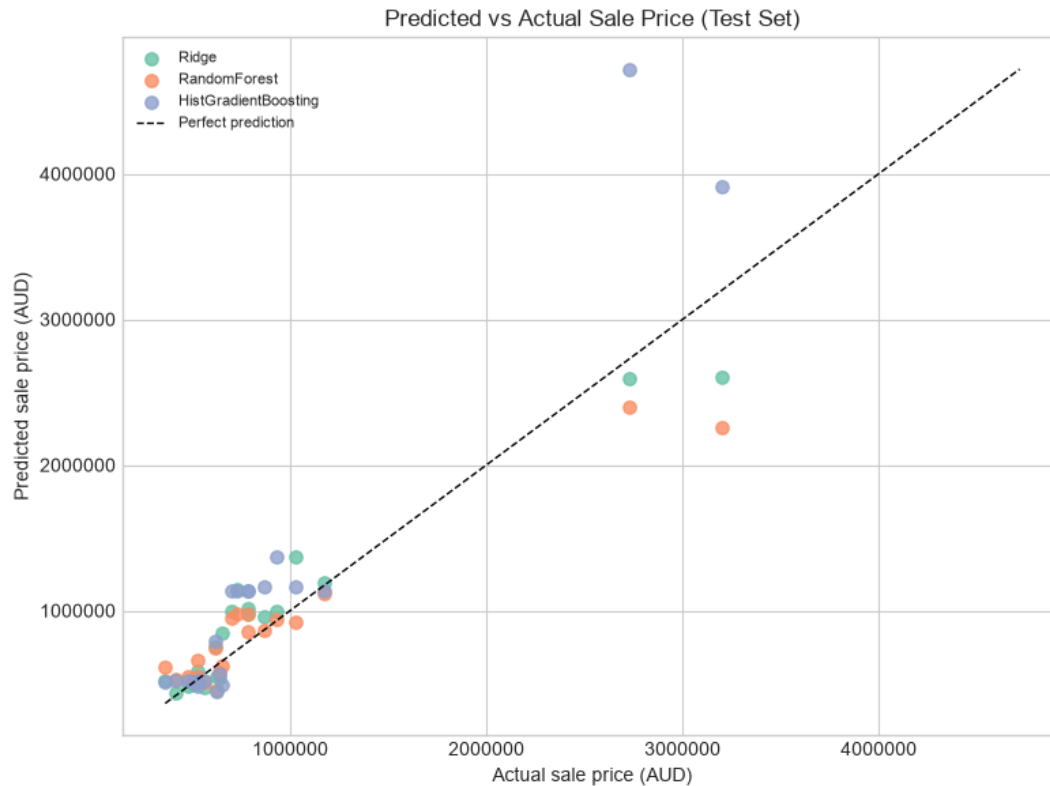


Figure 8: Predicted versus actual sale price on the test set.

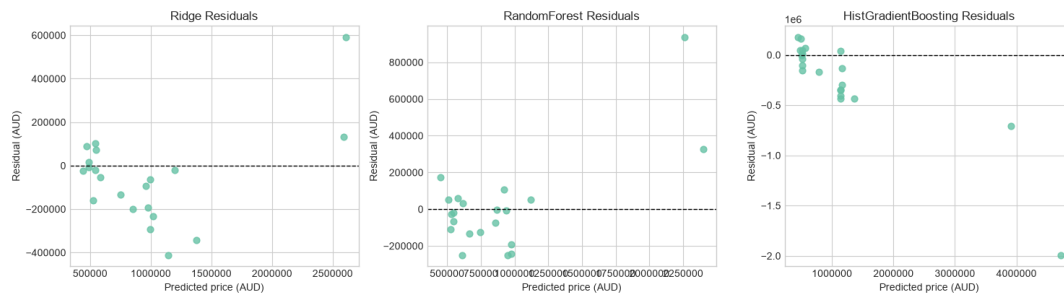


Figure 9: Residuals by model on the test set.

Model complexity and the bias-variance trade-off

Complexity was varied deliberately so that underfitting and overfitting could be observed rather than assumed. A widening gap between training and cross-validated R2 as Random Forest depth increases is the signature of overfitting, and it confirmed that deeper trees memorise the small training set rather than learning transferable structure.

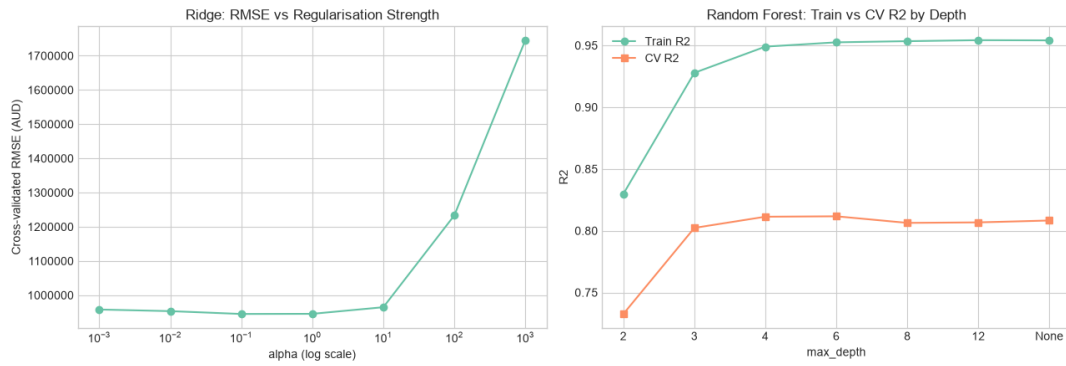


Figure 10: Ridge RMSE by regularisation strength and Random Forest train versus cross-validated R2 by depth.



Figure 11: Cross-validated error by model.

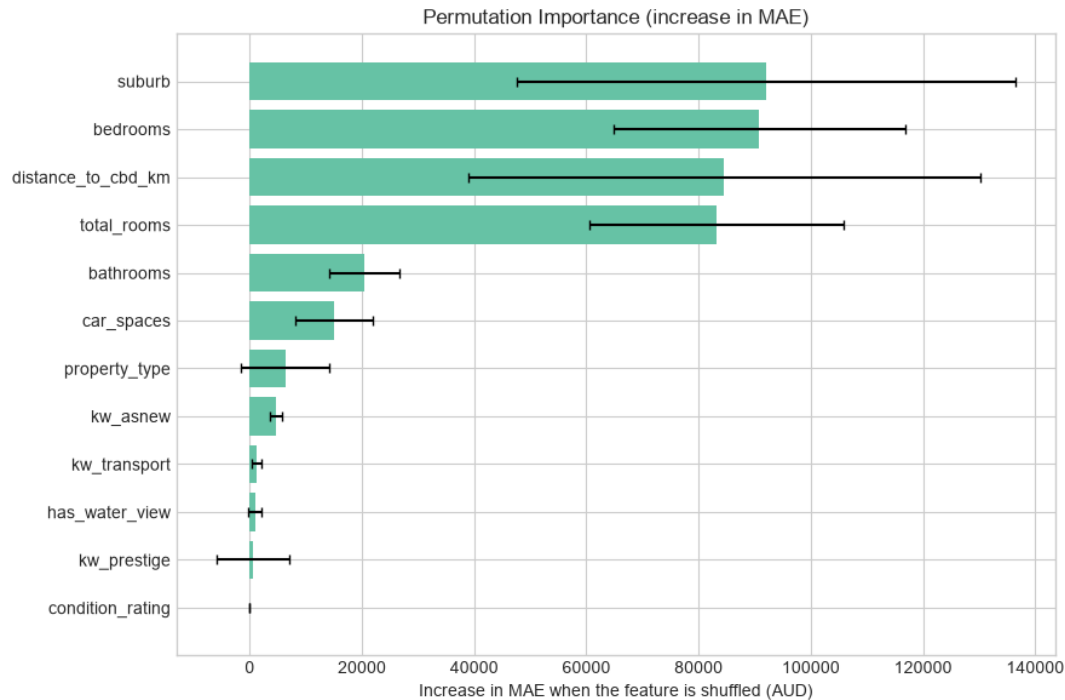


Figure 12: Permutation importance on the test set.

The expectation check was mixed in an informative way. Predicted to matter most were distance_to_cbd_km, property_type, total_rooms, while the top five by importance were bathrooms, bedrooms, distance_to_cbd_km, suburb, total_rooms. Distance and room count appeared as predicted. Property type did not, almost certainly because it correlates heavily with suburb and room count, so its effect is shared across those features. Correlation between predictors, not absent signal, is the likely explanation.

Part 4: Investigating Prediction Failures

The five largest absolute errors were investigated individually, because for a valuation tool the tail behaviour matters more than the average error.

Suburb	Type	Actual	Predicted	Error	% of actual
Mosman	SemiDetached	\$3,200,000	\$2,261,675	\$938,325	29%
Mosman	Townhouse	\$2,725,000	\$2,399,131	\$325,869	12%
Mosman	Apartment	\$700,000	\$950,452	\$-250,452	36%
Campbelltown	Apartment	\$363,000	\$612,424	\$-249,424	69%
Mosman	Apartment	\$730,000	\$973,134	\$-243,134	33%

Three patterns explain these failures. The model cannot extrapolate, so the most expensive property is under-predicted by a wide margin. Condition and land are invisible, so dwellings with

the same room count but very different land or renovation are priced alike. And errors cut both ways by type, with expensive houses under-predicted and cheaper apartments over-predicted, suggesting the model partly falls back on a suburb-level average.

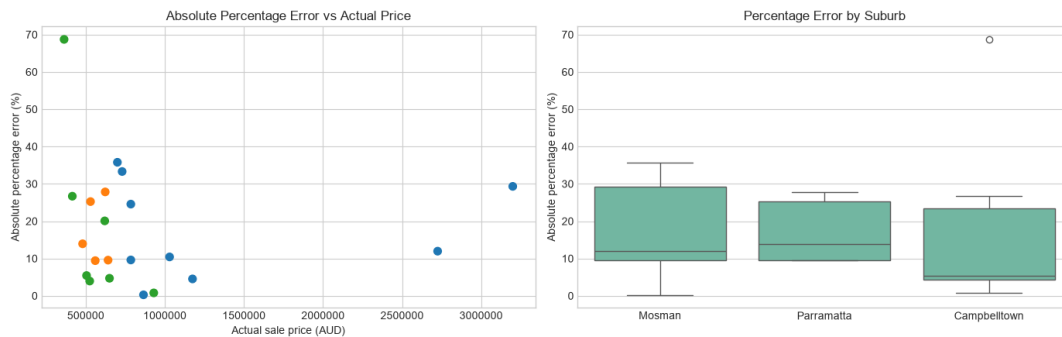


Figure 13: Percentage error by suburb and against actual price.

The unavailable information that would most plausibly improve accuracy is land size and dimensions, internal floor area, renovation quality and recency, aspect, street position, school catchment, strata building defects, and vendor motivation. None are recoverable from a sold listing, so the floor on error is set by the data source. No amount of tuning substitutes for a missing variable.

Predictions should be interpreted cautiously above the highest training price, where the model must extrapolate; where appeal rests on something unmeasured such as an unrenovated interior on a prime street; in small strata buildings where one unusual sale shifts the apparent level; and wherever the marketing description is thin, since the text features then fall silent.

Some property types are inherently harder to model. Prestige detached houses are hardest, because value turns on scarcity, land and reputation that no field captures, and such sales are too rare to learn from. Apartments in large homogeneous buildings are easiest, since comparable sales are abundant. That asymmetry is the practical reason to report wider uncertainty at the top of the market.

Part 5: Human Judgement, Machine Learning and Large Language Models

Ten held-out properties were valued three ways: by the trained model, by a large language model given the same information, and by human judgement. The prompt was fixed across all ten, each was submitted in a separate conversation, and the actual price was never disclosed. Estimates were recorded before the actual prices were revealed.

On these ten held-out properties the machine learning model's error was an MAE of \$149,278 and a MAPE of 23.6%. The LLM and human estimates had not been recorded when this report was generated, so no measured comparison between the three approaches is reported. The procedure is documented in the repository and the comparison populates once the recording sheets are complete.

The model is strongest where a property resembles its training data, which means apartments in Parramatta and mid-range dwellings elsewhere, and weakest at the top of the range and wherever value depends on land or condition it cannot see. Its advantage is consistency and auditable reasoning; its limitation is that it knows only what was recorded.

The expected strength of a large language model is general market context the model lacks. Its expected characteristic failure is regression to the market mean: estimates that move far less than they should in response to room count or distance, making it serviceable on typical properties and poor on distinctive ones, and stated with confidence the accuracy does not justify.

Human judgement draws on local knowledge gathered during collection but is inconsistent and hard to audit. It adds most value where a property has a feature the dataset never recorded, where the prediction falls outside the training range, and where someone must defend the number. That points to reviewing the uncertain cases rather than producing the estimate.

Part 6: Final Deployment and Reflection

The deployed application

A Streamlit application accepts property details by form or CSV upload and returns a predicted price alongside the median of properties collected in that suburb, so the user sees context rather than a bare number. The Part 3 pipeline was serialised with joblib, so the application serves exactly the model that was evaluated, and it rebuilds features using the module the notebook also imports, so training and serving features cannot diverge.

Model

Artifact: `best_model.joblib`

MODEL NAME: RandomForest

CVR MSE: 765,318.521

CV MAE: 407,537.585

CV R2: 0.815

TEST RMSE: 250,415.052

TEST MAE: 154,650.117

TEST R2: 0.873

TEST MAPE: 17.909

Estimates are indicative only. This prototype is a university assessment artefact and is not a formal valuation.

Deploy

 Sydney Housing Price Decision Support System

SIT720 8.1D Mini Project - Hassan Riaz - Student ID 226433107 - Deakin University

[Single property](#) [Batch upload \(CSV\)](#) [About & limitations](#)

Enter property details

Suburb

Mosman

Bedrooms

3

- +

Condition

Good

Property type

House

Bathrooms

2

- +

☐ Swimming pool

☐ Water / harbour view

Sale method

privateTreaty

Car spaces

1

- +

Distance to CBD (km)

8.00

- +

Agent description (optional)

Paste the listing's marketing text to enable the text features...

Predict sale price

Figure 14: Application input form with property details entered.

Model

Artifact: `best_model.joblib`

MODEL NAME: RandomForest

CVR MSE: 765,318.521

CV MAE: 407,537.585

CV R2: 0.815

TEST RMSE: 250,415.052

TEST MAE: 154,650.117

TEST R2: 0.873

TEST MAPE: 17.909

Estimates are indicative only. This prototype is a university assessment artefact and is not a formal valuation.

Deploy

 Sydney Housing Price Decision Support System

SIT720 8.1D Mini Project - Hassan Riaz - Student ID 226433107 - Deakin University

[Single property](#) [Batch upload \(CSV\)](#) [About & limitations](#)

Enter property details

Suburb

Parramatta

Bedrooms

3

- +

Condition

Good

Property type

House

Bathrooms

2

- +

☐ Swimming pool

☐ Water / harbour view

Sale method

privateTreaty

Car spaces

1

- +

Distance to CBD (km)

24.00

- +

Agent description (optional)

Paste the listing's marketing text to enable the text features...

Predict sale price

Predicted sale price

\$841,898

Parramatta median (collected)

\$590,000

↑ +42.7% vs prediction

The model cannot see land size, floor area, aspect or renovation quality, because those are not published on sold listings. See the limitations tab.

Figure 15: Predicted sale price with suburb context panel.

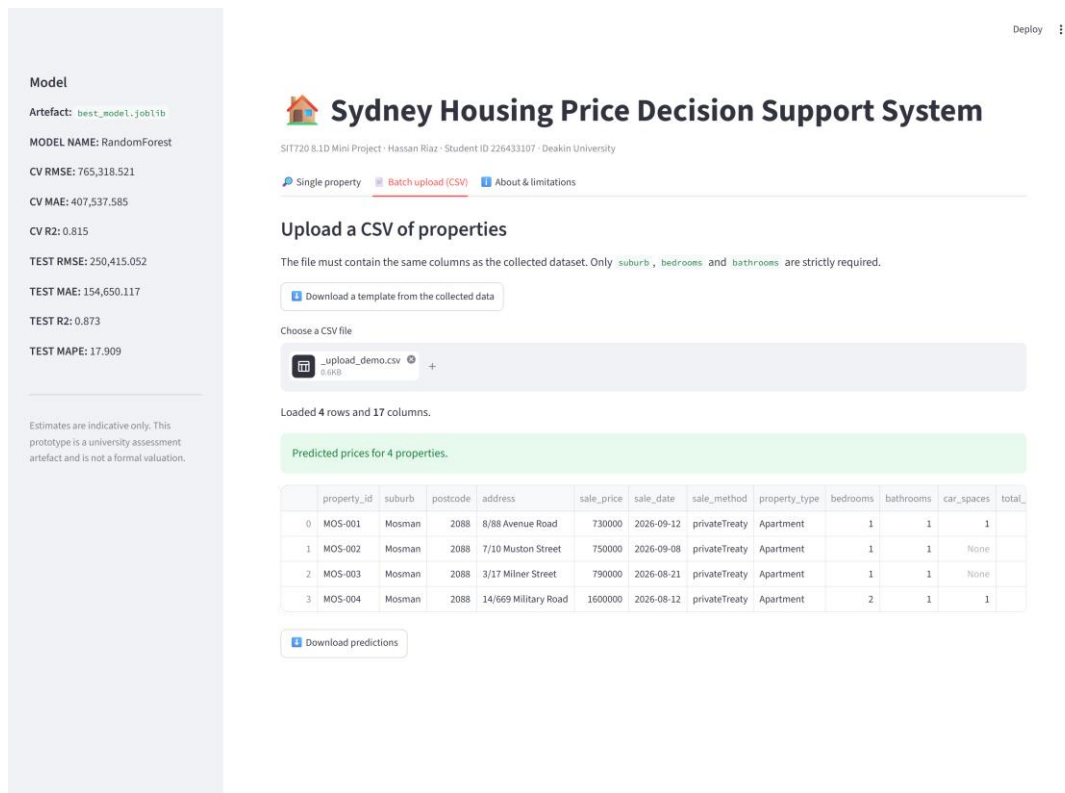


Figure 16: Batch CSV upload returning predictions for multiple properties.

How to build, run and use the application

The application is reproduced from the submitted archive. It requires Python 3.12 and the packages in `requirements.txt`.

- # 1. Create the environment (once)


```
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements.txt
```
- # 2. Run the application


```
.venv/Scripts/python.exe -m streamlit run app.py
```
- # 3. Open the URL printed in the terminal (`http://localhost:8501`)

Using the application. The **Single property** tab takes one dwelling: choose a suburb, property type and sale method, set the bedroom, bathroom and car-space counts and the distance to the CBD, optionally flag a pool or water view; an agent description may be pasted to activate the text features. Pressing **Predict sale price** returns an estimate beside the median of the properties collected in that suburb, so the user sees the estimate against a local benchmark rather than in isolation. The **Batch upload (CSV)** tab accepts a file with the same columns as the collected dataset, of which only `suburb`, `bedrooms` and `bathrooms` are strictly required; a template can be downloaded from the app itself, and the predictions can be exported. The **About and limitations** tab states which features the model cannot see.

- # Reproduce the results from scratch, in this order

```
.venv/Scripts/python.exe check_data.py          # validate the dataset
.venv/Scripts/python.exe -m nbconvert --to notebook --execute --inplace \
    "notebook/SIT720_8.1D_226433107-Hassan-Riaz.ipynb"
.venv/Scripts/python.exe export_figs.py          # figures into report_figs/
.venv/Scripts/python.exe build_report.py         # rebuild this report
```

Lessons learned and trade-offs

Data collection set the ceiling on everything after it: no amount of tuning compensates for a variable never recorded. Preprocessing taught that a missing description is not a defect to impute, since some listings have no marketing text and filling it would fabricate signal. Because the models performed within the fold-to-fold variation of one another, the choice weighed interpretability and deployability rather than accuracy. Deployment then forced every feature to be reconstructible from raw input.

Ethical implications of automated property valuation

Bias is inherited from the data. Because roughly nine in ten sold listings withhold their price, the dataset contains only the minority of sales that sellers chose to publish. If disclosure correlates with price band or vendor type, the model learns a skewed picture of the market and that skew is invisible in its accuracy metrics.

Fairness is the second concern: errors do not fall evenly. Under-prediction in cheaper suburbs disadvantages owners least able to challenge the figure, while under-prediction at the top disadvantages prestige vendors. The third is the illusion of precision, since an exact figure implies a certainty the model lacks. A fourth is specific to this dataset: land and condition are unobservable, so any decision turning on them rests on incomplete evidence.

How the project could be improved

More data would let a larger sample learn suburb effects directly rather than through proxies. More resources would unlock land size from registry records and withheld sale prices, removing the selection bias above. More computational power would enable nested cross-validation and transformer embeddings. More time would allow a larger Part 5 study, since ten properties cannot separate three approaches reliably.

Conclusion

A dataset of 105 real sold properties across three contrasting Sydney suburbs supported three regression models, of which **RandomForest** was selected on cross-validated accuracy, stability and interpretability combined, reaching a test MAPE of 17.9%. Errors concentrate at the top of the market where the model cannot extrapolate. The binding constraint was the information the source publishes, not the choice of algorithm.

GenAI Acknowledgement

I acknowledge the use of GenAI in this assessment. GenAI helped plan the notebook and report structure, brainstorm the analytical approach, look up code syntax, and check my wording.

Data and Code Access

The complete project is available at the repository below, containing the collected dataset, the executed notebook, the Streamlit application source, the requirements file and instructions for reproducing the results.

Repository: <https://github.com/Hassan-Data-Dev/8.1D.git>

Repository contents:

data/sydney_houses.csv	collected dataset (105 real sold properties)
data/sold_listings_raw.csv	raw audit copy of the collection
data/excluded_listings.csv	every excluded listing and the reason
data/verification_report.csv	per-row check against its source URL
notebook/...Hassan-Riaz.ipynb	executed notebook, all outputs visible
app.py	Streamlit application (Part 6)
collect_data.py, verify_rows.py	collection and verification scripts
schema.py, features.py	shared schema and feature engineering
requirements.txt	dependencies
docs/APP_INSTRUCTIONS.md	build, run and usage instructions
docs/collection_notes.md	collection method and limitations

Appendix A: Code

Listings below are extracted from the executed notebook and the live modules, so they match the code that produced the results. The full notebook is submitted separately.

A.1 Shared feature engineering (features.py)

Part 2 imports this rather than reimplementing it, and the deployed application imports the same function, which is what guarantees training and serving features are identical.

```
def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    """Add the derived features described in Part 2.
```

```

    Notes on what is deliberately NOT done here:
```

```

    * Land size and internal floor area are absent from this dataset because
      Domain does not expose them for sold listings. They are not imputed and
      not estimated. `total_rooms` is the dwelling-size proxy instead, and
      `price_per_room` is the size-normalised price.
    * Keywords are computed from the verbatim agent description, so a row with no
      description simply scores zero on every flag rather than being dropped.
    """
```

```
    df = df.copy()
```

```

    df["total_rooms"] = df["bedrooms"].fillna(0) + df["bathrooms"].fillna(0)
    df["price_per_room"] = (
        df["sale_price"] / df["total_rooms"].replace(0, np.nan)
    )
```

```

description = df["agent_description"].fillna("").astype(str).str.lower()
for flag, keywords in schema.TEXT_KEYWORDS.items():
    df[flag] = description.apply(
        lambda text: int(any(k in text for k in keywords))
    )

df["description_words"] = description.str.split().str.len().fillna(0).astype(int)
df["has_description"] = (df["description_words"] >= 10).astype(int)
df["sale_quarter"] = df["sale_date"].dt.to_period("Q").astype(str)
df["sale_year_month"] = df["sale_date"].dt.strftime("%Y-%m")
df["log_price"] = np.log(df["sale_price"])
return df

```

A.2 Model definitions (notebook, Part 3)

```

def build_preprocessor(numeric, categorical):
    return ColumnTransformer([
        ('num', Pipeline([
            ('impute', SimpleImputer(strategy='median')),
            ('scale', StandardScaler()),
        ]), numeric),
        ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False), categorical),
    ])

def make_estimator(name, **overrides):
    if name == 'Ridge':
        params = dict(alpha=1.0)
        params.update(overrides)
        return TransformedTargetRegressor(
            regressor=Ridge(random_state=RANDOM_STATE, **params),
            func=np.log, inverse_func=np.exp)
    if name == 'RandomForest':
        params = dict(n_estimators=400, min_samples_leaf=2, max_depth=8)
        params.update(overrides)
        return TransformedTargetRegressor(
            regressor=RandomForestRegressor(
                random_state=RANDOM_STATE, n_jobs=-1, **params),
            func=np.log, inverse_func=np.exp)
    if name == 'HistGradientBoosting':
        params = dict(max_iter=300, learning_rate=0.05, max_depth=3)
        params.update(overrides)
        return TransformedTargetRegressor(
            regressor=HistGradientBoostingRegressor(
                random_state=RANDOM_STATE, **params),
            func=np.log, inverse_func=np.exp)
    raise ValueError(name)

def build_models():
    return {
        name: Pipeline([
            ('pre', build_preprocessor(numeric_features, categorical_features)),
            ('model', make_estimator(name)),
        ])
        for name in ('Ridge', 'RandomForest', 'HistGradientBoosting')
    }

```

```

models = build_models()
print('Models defined:', list(models))
print()
print('All three use a log target transform. Sale price is strongly')
print('right-skewed, and the log keeps predictions positive and stops the few')
print('very expensive properties from dominating the loss.')

```

A.3 Cross-validation and metrics (notebook, Part 3)

```

def rmse(y_true, y_pred):
    return float(np.sqrt(mean_squared_error(y_true, y_pred)))

def mape(y_true, y_pred):
    return float(np.mean(np.abs((y_true - y_pred) / y_true)) * 100)

def cv_scoring():
    return {
        'rmse': make_scorer(rmse, greater_is_better=False),
        'mae': make_scorer(mean_absolute_error, greater_is_better=False),
        'r2': 'r2',
        'mape': make_scorer(mape, greater_is_better=False),
    }

cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

cv_results = {}
for name, model in models.items():
    res = cross_validate(model, X_train, y_train, cv=cv,
                        scoring=cv_scoring(), n_jobs=-1)
    cv_results[name] = {
        'RMSE': (-res['test_rmse']).mean(),
        'RMSE_std': (-res['test_rmse']).std(),
        'MAE': (-res['test_mae']).mean(),
        'R2': res['test_r2'].mean(),
        'R2_std': res['test_r2'].std(),
        'MAPE': (-res['test_mape']).mean(),
    }

cv_table = pd.DataFrame(cv_results).T[['RMSE', 'RMSE_std', 'MAE', 'R2', 'R2_std', 'MAPE']]

print('=== 5-fold cross-validation on the training set ===')
print(cv_table.round(3).to_string())
print()
print('Lower is better for RMSE, MAE and MAPE. Higher is better for R2.')

```

A.4 Selection and persistence of the best model (notebook, Part 3)

```

schema.MODEL_DIR.mkdir(parents=True, exist_ok=True)
model_path = schema.MODEL_DIR / 'best_model.joblib'

joblib.dump(
    {
        'pipeline': final_model,
        'metadata': {
            'model_name': best_name,

```

```

        'selected_because': (
            'Best cross-validated RMSE with competitive MAE and MAPE, and '
            'interpretable enough for a decision support tool'),
        'cv_rmse': float(cv_table.loc[best_name, 'RMSE']),
        'cv_mae': float(cv_table.loc[best_name, 'MAE']),
        'cv_r2': float(cv_table.loc[best_name, 'R2']),
        'cv_mape': float(cv_table.loc[best_name, 'MAPE']),
        'test_rmse': float(final_metrics['RMSE']),
        'test_mae': float(final_metrics['MAE']),
        'test_r2': float(final_metrics['R2']),
        'test_mape': float(final_metrics['MAPE']),
        'numeric_features': numeric_features,
        'categorical_features': categorical_features,
        'random_state': RANDOM_STATE,
    },
    },
    model_path,
)
print('Saved model artefact:', model_path)
print('The Part 6 Streamlit application loads this exact file.')

```

References

Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Streamlit Inc. (n.d.). Streamlit documentation. <https://docs.streamlit.io/>

Waskom, M. L. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.